CS163: Data structures

Topic: Treap

2026

Today?

- **Continue Discussing Tree**
- Examine more advanced trees
  - ~~*2-3 (evaluate what we learned)*~~
  - ~~*B-Trees*~~
  - ~~*AVL*~~
  - ~~*2-3-4*~~
  - ~~*red-black trees*~~
  - Treap

Treap

- **Treap** is a ~~*self-balancing*~~ **randomized** *binary search tree*.

Introduction

- A randomized binary search tree:
  - Designed to maintain ordered data
  - Avoids worst-case ***O(N)*** depth of BST
  - Uses random priorities to stay balanced

Introduction

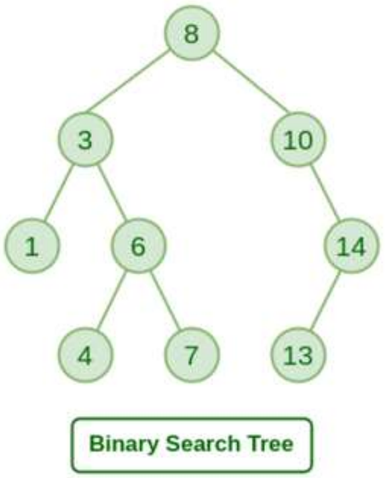
- Treap = Binary Search Tree + Heap

Treap Properties

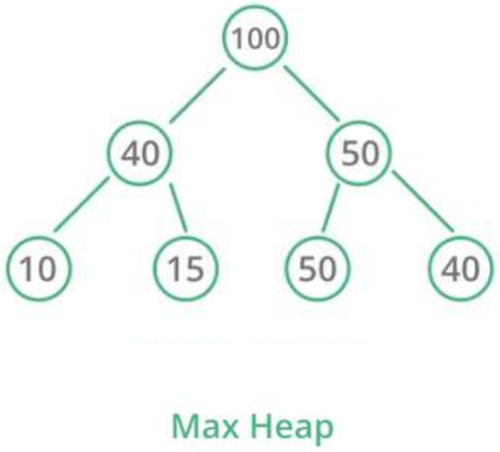
- Satisfies two distinct properties simultaneously:
  - Binary Search Tree Property:
    - left subtree keys < node key
    - right subtree keys > node key
  - Heap property (max - heap):
    - parent priority ≥ children priorities

Treap Properties

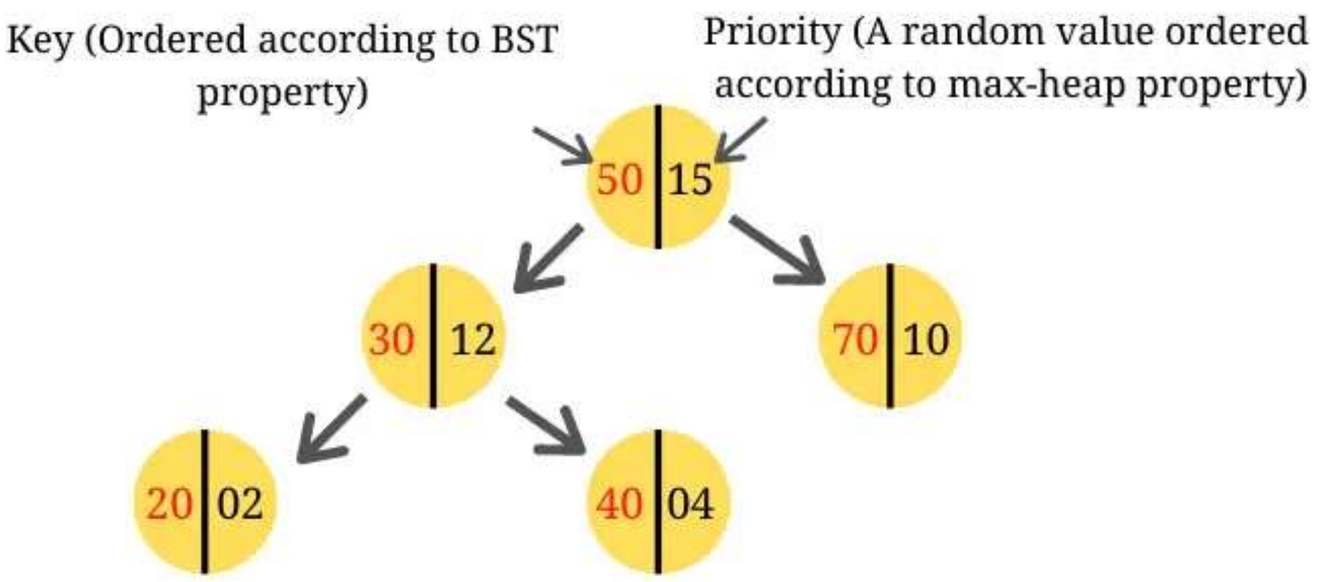
Binary Search Tree



Heap



Treap Properties



Why Treap?

- Each node gets a random priority
- Tree shape becomes random
- Similar to a random binary tree

History of Treap

- Proposed by Seidel & Aragon (1989)
- Published in Algorithmica (1996)
- Later work:
  - Randomized BST (Martínez & Roura)
  - Applications in cryptography

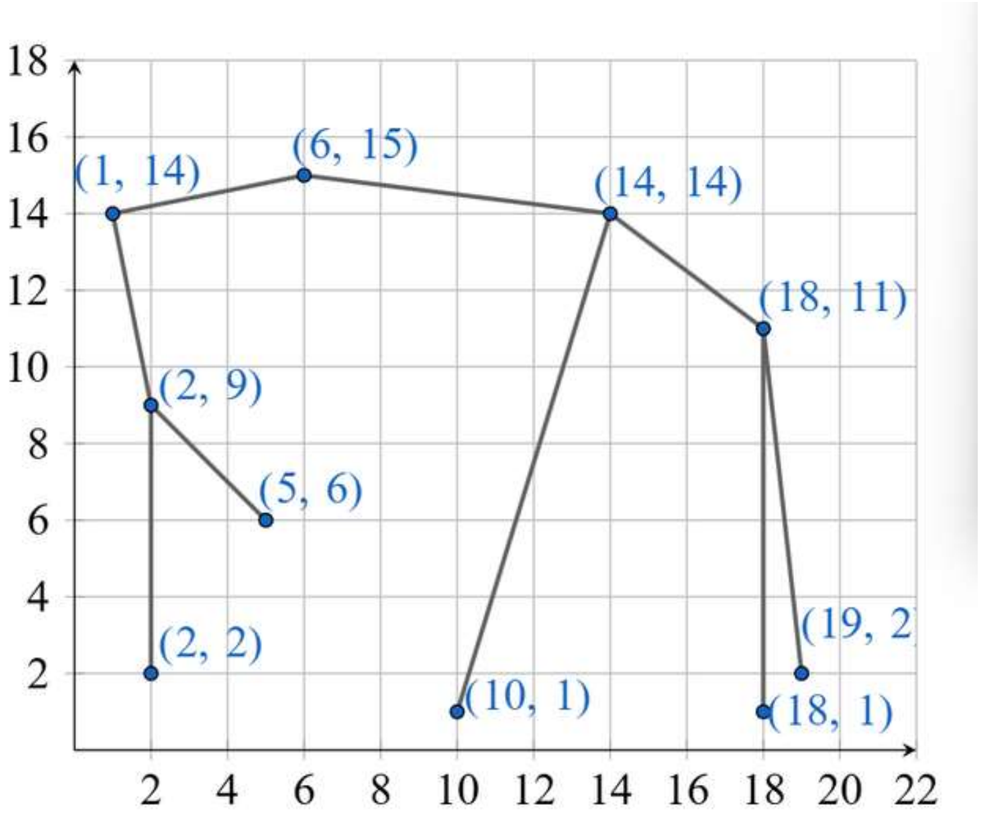
History of Treap

- Connections:
  - Similar to Quicksort randomness
  - Based on Cartesian Tree (1980)

History of Treap

- ***<u>Quiz</u>***: Why is treap also called Cartesian tree?

History of Treap



Main Data Structures

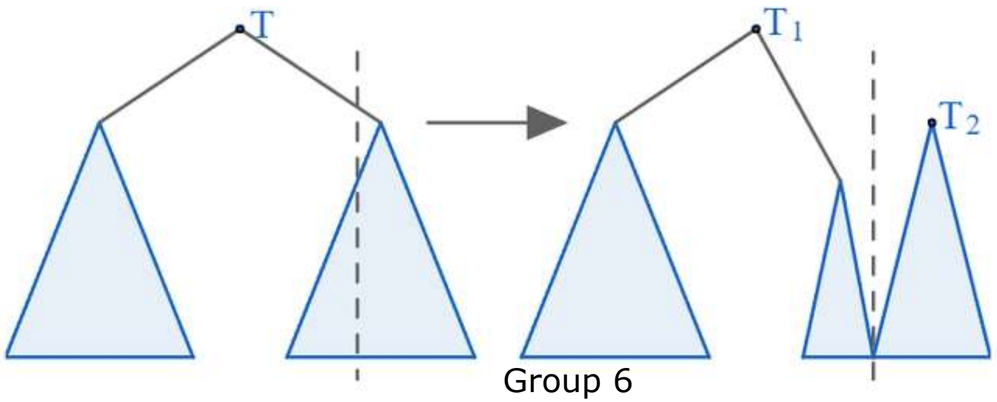
- Stores key + priority
- Works like BST + Heap

Main Algorithms in Treap

- Most treap's operations are built from split and merge:
  - **Split**: Divide tree into 2 parts
  - **Merge**: Combine two trees into one
- This is a key feature that makes Treap simple and flexible

Split Operation

- Splits one treap into two valid treaps:
  - Left treap contains keys ≤ k
  - Right treap contains keys > k



Split Operation

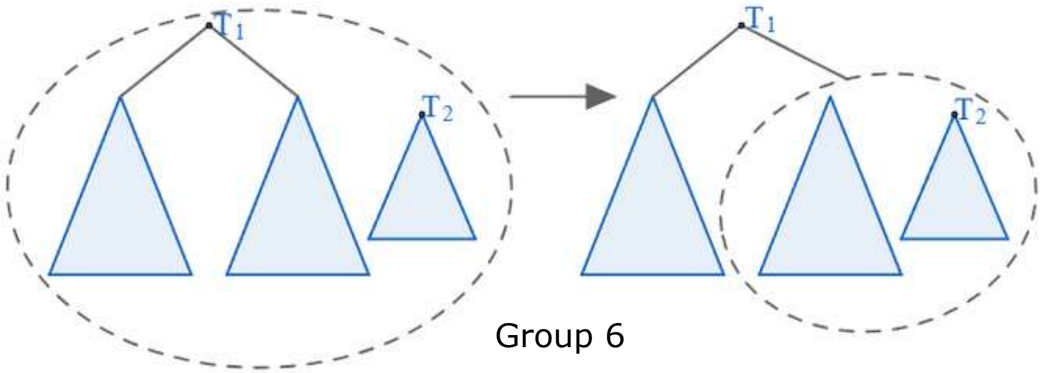
- Unlike AVL or Red-Black trees, Treap supports split naturally
- This makes many advanced operations much easier

Split Operation

- Example

Merge Operation

- Combines two treaps into one
- Precondition:
  - Every key in left treap is smaller than every key in right treap
  - Root is chosen by comparing priorities
  - Recursively connects the remaining subtrees



Merge Operation

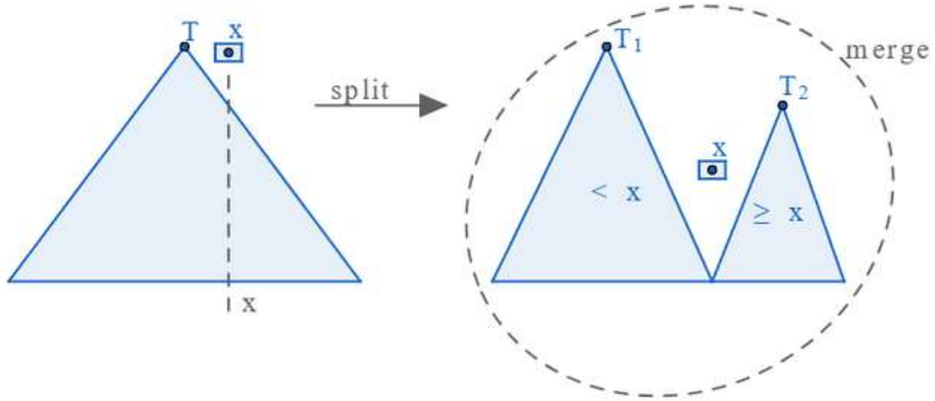
- Merge is a built-in structural operation of Treap
- This is much harder to do directly in AVL or Red-Black trees

Merge Operation

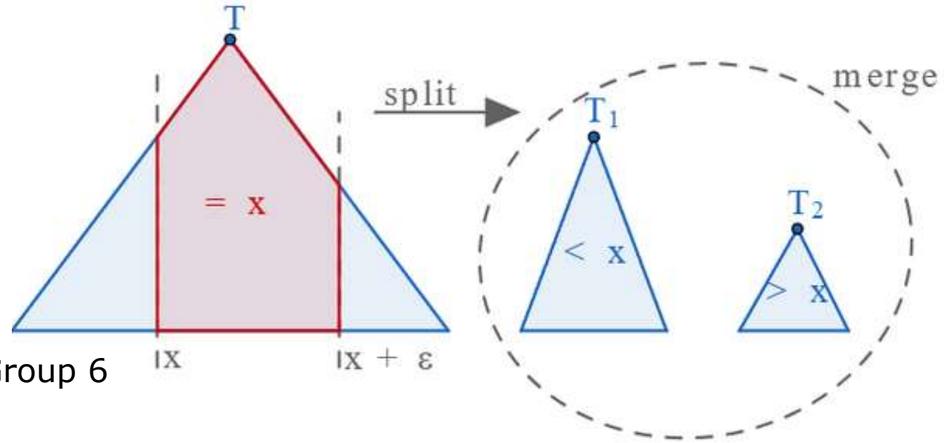
- Example

Insert and Erase

- **Insert**: split → add node → merge



- **Erase**: split → remove node → merge



Bulk Set Operations

- Treap also supports efficient set operations:
  - Union
  - Intersection
  - Difference

Bulk Set Operations

- Strategy:
  - choose the root with higher priority
  - split the other treap by that root key
  - recursively combine corresponding subtrees
- Works efficiently on whole sets, not just single elements

Why Treap is Special

- Easier to extend for:
  - Set operations
  - Range operations
  - Implicit sequence structures
- Simpler and more flexible than many self-balancing trees

Why Treap is Special

- BST: simple, but may become unbalanced
- AVL / Red-Black: balanced, but more complex
- Treap: randomized, elegant, and highly flexible

Implementing Treap

- Treap Node:

```
struct Node{
    int val;
    int pri;
    Node *left, *right;

    Node(int k){
        val = k;
        pri = rng();
        left = right = nullptr;
    }
};
```

Implementing Treap

- Random Number?

Implementing Treap

- Some built-in random number generators:
  - C++:
    - `#include <random>`
    - `rand()`
    - `mt19937`
  - Python:
    - `import random`
    - `randint()`
  - `...`

Implementing Treap

- Splitting

Implementing Treap

```
void split(Node* root, int val, Node* &left, Node* &right){
  if (root == nullptr){
    left = right = nullptr;
    return;
  }

  if (root->val <= val){
    split(root->right, val, root->right, right);
    left = root;
  } else{
    split(root->left, val, left, root->left);
    right = root;
  }
}
```

Implementing Treap

- Merging

Implementing Treap

```
Node* merge(Node* left, Node* right) {
  if (left == nullptr || right == nullptr){
    return left ? left : right;
  }

  if (left->pri > right->pri) {
    left->right = merge(left->right, right);
    return left;
  } else {
    right->left = merge(left, right->left);
    return right;
  }
}
```

Implementing Treap

- Inserting

Implementing Treap

```
void insert(Node* &root, int val){
  Node *left = nullptr;
  Node *mid = nullptr;
  Node *right = nullptr;

  split(root, val, left, right);
  split(left, val - 1, left, mid);

  if (mid == nullptr){
    mid = new Node(val);
  }

  root = merge(merge(left, mid), right);
}
```

Implementing Treap

- Erasing

Implementing Treap

```
void erase(Node* &root, int val){
  Node *left = nullptr;
  Node *mid = nullptr;
  Node *right = nullptr;

  split(root, val, left, right);
  split(left, val - 1, left, mid);

  if (mid != nullptr){
    delete mid;
  }

  root = merge(left, right);
}
```

Complexity?

- How about the **Complexity**?
- Isn’t it ***O(N)*** splitting and merging?

Complexity?

- Every node in a Treap contains 2 values:
  - **Key:** satisfies the property of a **BST**
  - **Priority:** assigned by a ***random*** value and satisfies the property of the **Heap.**

Complexity?

- The major issue of a normal **BST** is that the worst complexity can be up to ***O(N)*** if we insert the data in a sorted order.

- But in a **Treap**, by assigning ***priority*** randomly, the order of the nodes no longer depends on the order that we input.

Complexity?

- A **BST** that created by a random permutation of ***N*** element has the expected height is ***O(logN)***

- Moreover, by *probability and statistics*, the average distance between the root to a random node inside the **Treap** is approximately ***2lnN***

Complexity?

- Therefore, the **<u>average complexity</u>** of the operations (like finding, inserting, erasing,...) is ***O(logN)***

Complexity?

<u>Quiz</u>: What is the worst complexity of the Treap?

Implicit Treap

- In their most basic form, **treaps** are not really useful, because we have many ***self-balancing trees*** that are much more efficient and faster than **treaps.**

- However, with **implicit treaps**, we can efficiently perform operations on a regular array in fashion similar to ***segment trees*** and ***fenwick trees***.

Implicit Treap

- Some operations can be done by **implicit treaps**:
  - Insert an element ***x*** at position ***i***
  - Delete the element at position ***i***
  - Performing interval queries (**sum**, **min**, **max**, etc.)
  - Perform interval updates (**add**, **set**, **reverse**, etc.)

Implicit Treap

- Normal Treap Node:

```
struct Node{
    int val;
    int pri;
    Node *left, *right;

    Node(int k){
        val = k;
        pri = rng();
        left = right = nullptr;
    }
};
```

Implicit Treap

- Implicit Treap Node

```
struct Node{
    int val;
    int pri;
    int size;
    Node *left, *right;

    Node(int k){
        val = k;
        pri = rng();
        size = 1;
        left = right = nullptr;
    }
};
```

Implicit Treap

- Splitting

Implicit Treap

```
void split(Node* root, int k, Node* &left, Node* &right){
  if (root == nullptr){
    left = right = nullptr;
    return;
  }

  [...]
}
```

Implicit Treap

```
int idx = nodeSize(root->left);
if (idx < k){
   split(root->right, k - idx - 1, root->right, right);
   left = root;
} else{
   split(root->left, val, left, root->left);
   right = root;
}

root->size = 1 + nodeSize(root->left) + nodeSize(root->right);
```

Implicit Treap

- Merging is the same with a normal **treap.**

Implicit Treap

- A node in a **Implicit Treap** represents a **segment** of an array.
- Combine with the power of ***splitting*** and ***merging*** of a **Treap,** we can **cut**, **paste,** and make other operations with the **segments** of the arrays by using this data structure.

Application

- Standard Treap:
  - **Ease of implementation:** It is easy to implement more than other ***self-balancing trees.***
  - **Ordered set:** By storing the size of the subtree at each node, we can find the ***kth-smallest element*** in ***O(logN)***

Application

- Standard Treap:
  - **Joining and Splitting Databases:** Treaps can be split into two trees and merged back together in ***O(logN)*** time.

Application

- Implicit Treap:
  - **Dynamic Array with Fast Insert/Delete**
  - **Persistent Data Structures:** Useful in **Undo** mechanisms in text editors or version control logic.
  - **Rope Data Structure:** used to manage large blocks of text, with efficient cutting, copying, and pasting massive strings.

Programming Assignments

Reversals and Sums (CSES 2074)

Given an array of *n* integers, you have to process following operations:

1. reverse a subarray
2. calculate the sum of values in a subarray

Programming Assignments

- Solution: **Implicit Treap**

Programming Assignments

- Each node stores an extra element **sum**

Programming Assignments

- Calculate the sum (2nd operation):
  1. Split the treap into 3 parts:
     - **[1, L - 1]**
     - **[L, R]**
     - **[R + 1, N]**
  2. Output the sum of the middle part
  3. Merge

Programming Assignments

- Reverse a subarray: **Lazy propagation**
- Each node stores an extra flag **lazy** to check if that segment is reversed or not

Programming Assignments

- When we reach a node that has its **lazy** is *true*, we swap its left and right child node, update the **lazy** of its children, then we set its **lazy** to *false* again.
- This process can be solved simply by using ***XOR (Exclusive OR)***

Programming Assignments

- Reverse a subarray:
  1. Split the treap into 3 parts:
     - **[1, L - 1]**
     - **[L, R]**
     - **[R + 1, N]**
  2. Update the **lazy** of the middle Node:
     - `mid->lazy ^= 1`
  3. Merge

Programming Assignments

Substring Reversals (CSES 2073)

Given a string, your task is to process operations where you reverse a substring of the string. What is the final string after all the operations?

Programming Assignments

Game (IOI 2013)

You need to manage a massive 2D grid of size $R \times C$ (up to $10^9 \times 10^9$). Initially, all cells contain the value 0. You must process N queries ($N \leq 22,000$) of two types:

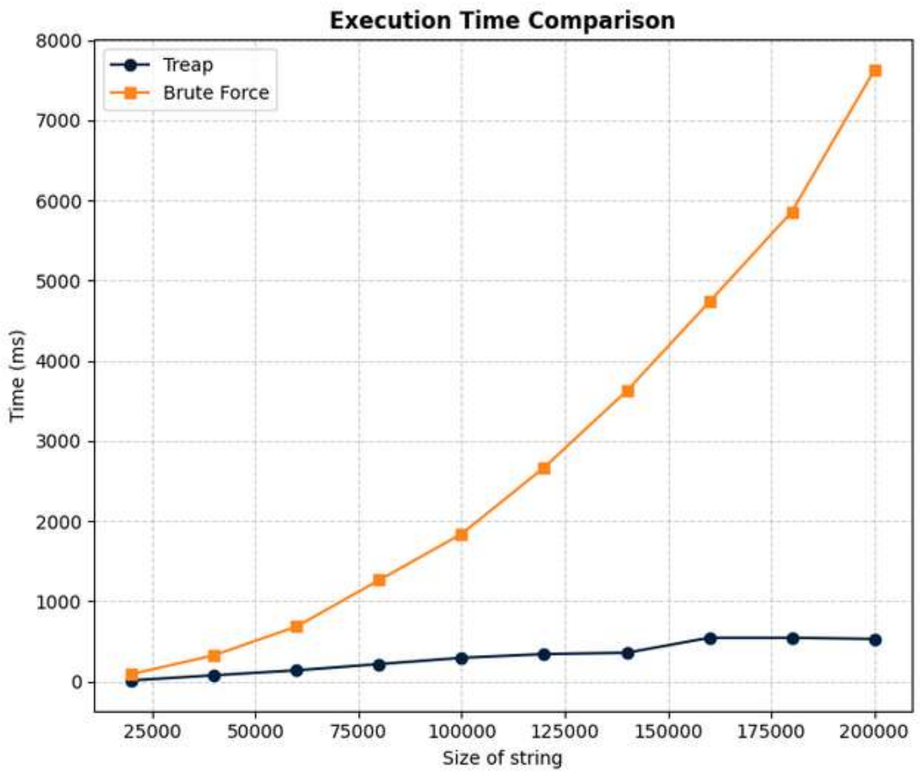
- **Update:** Assign a non-negative integer K to the cell at row P, column Q.
- **Calculate (Query):** Find the Greatest Common Divisor (GCD) of all cells within a subgrid defined by the top-left corner (P, Q) and the bottom-right corner (U, V).

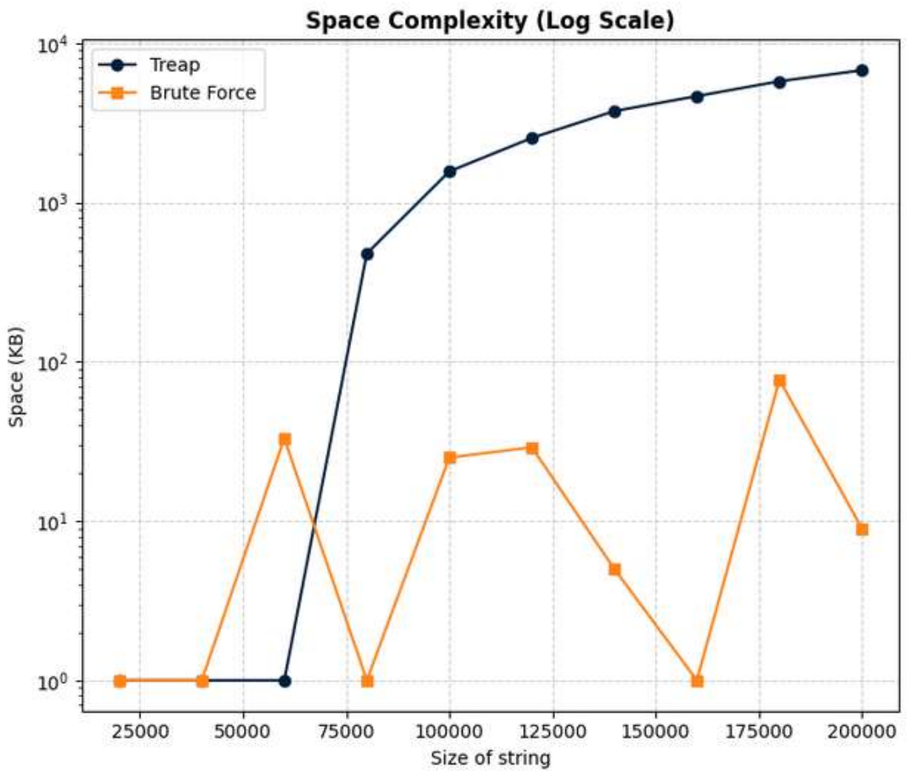
Comparison

Benchmark Problem

- **Substring Reversals (CSES 2073)**
  - Perform many range operations [a, b]
  - Reverse substring in place
  - Input size up to 200,000

Result:


Execution Time Comparison
Treap
Brute Force
Time (ms)
Size of string
0
1000
2000
3000
4000
5000
6000
7000
8000
25000
50000
75000
100000
125000
150000
175000
200000


Space Complexity (Log Scale)
Treap
Brute Force
Space (KB)
Size of string
10^0
10^1
10^2
10^3
10^4
25000
50000
75000
100000
125000
150000
175000
200000

Performance Results

- **Brute Force:**
  - Time: ***O(nm)*** as the string size (***n***) and operations (***m***) (reach **7500ms**)
  - Space: Maintains a very low and inconsistent memory footprint (under **100 KB**)
- **Treap:**
  - Time: ***O(m log n)*** (reach **600ms**)
  - Space: Reaching nearly **10,000 KB** (**10 MB**).

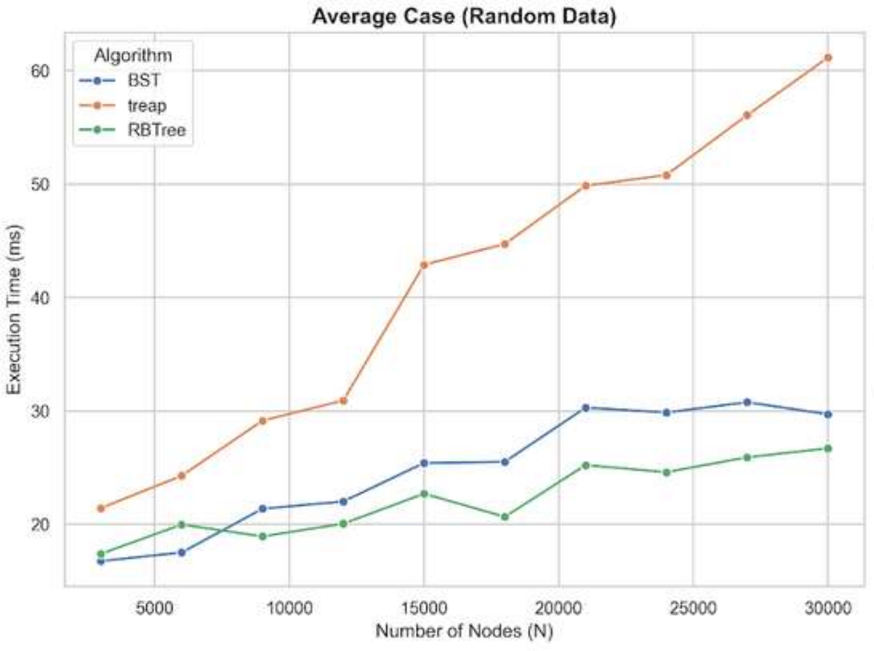
Why Treap Works Better

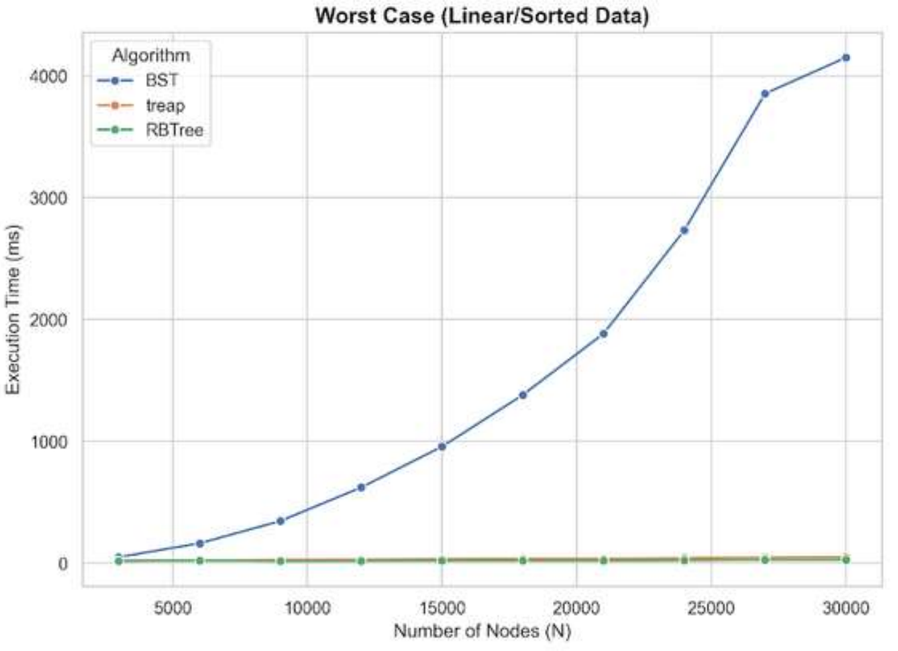
- Supports range operations efficiently
- Avoids repeated string copying
- Time complexity: ***O(m log n)*** vs ***O(nm)***

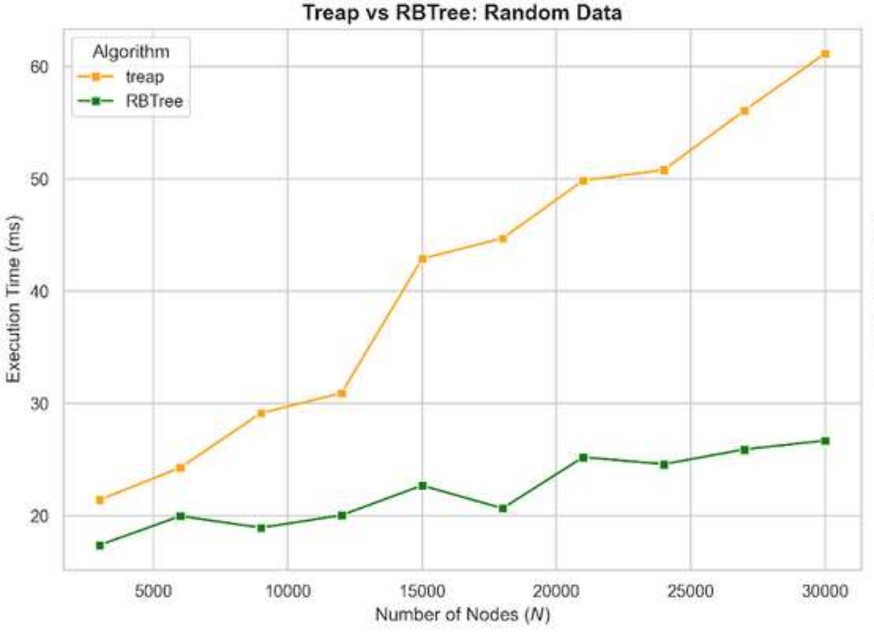
- *“Treap transforms the problem into structural operations instead of repeated modifications.”*

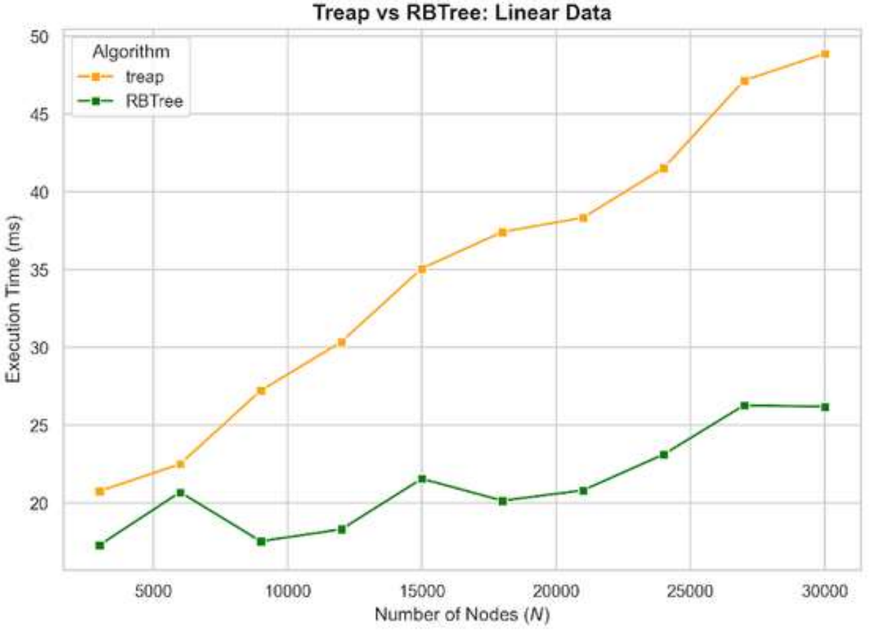
Second Experiment

- Insert and retrieve elements
- Compare:
  - BST
  - Treap
  - Red-Black Tree
- Test on:
  - random data
  - sorted data


Average Case (Random Data)
Algorithm
BST
treap
RBTree
Execution Time (ms)
Number of Nodes (N)


Worst Case (Linear/Sorted Data)
Algorithm
BST
treap
RBTree
Execution Time (ms)
Number of Nodes (N)

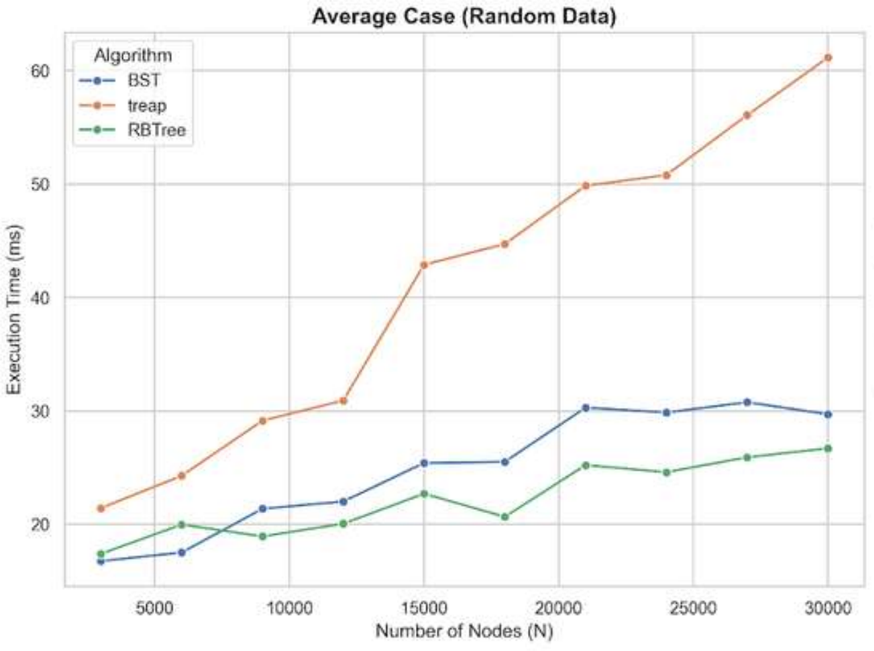

Treap vs RBTree: Random Data
Algorithm
treap
RBTree
Execution Time (ms)
Number of Nodes (N)

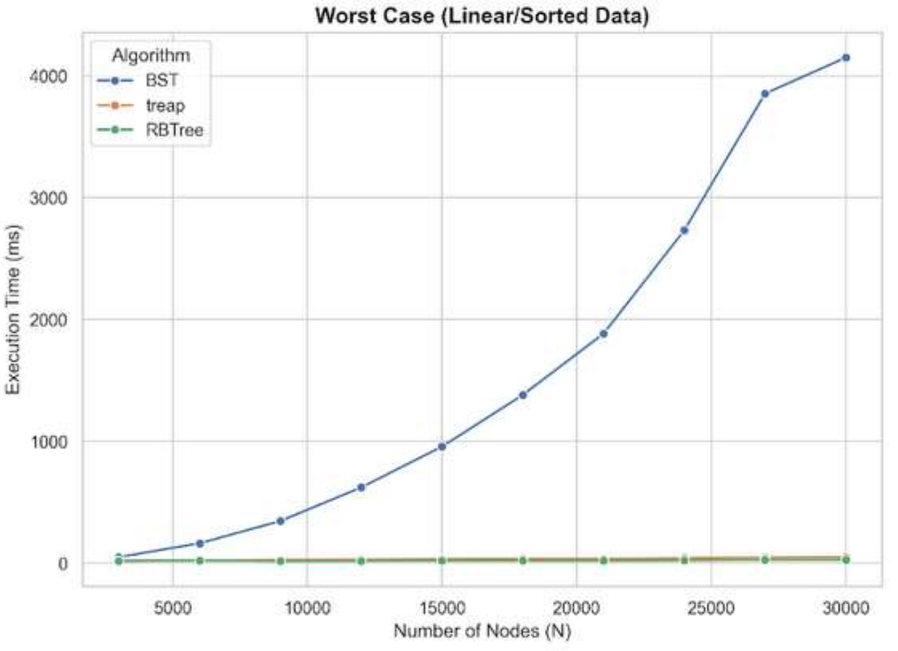

Treap vs RBTree: Linear Data
Algorithm
treap
RBTree
Execution Time (ms)
Number of Nodes (N)

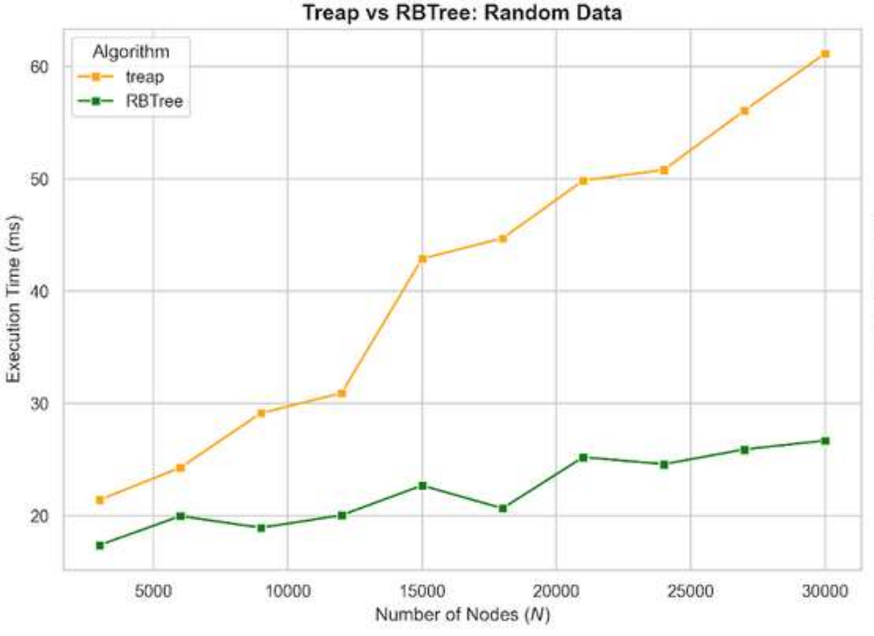
BST Performance

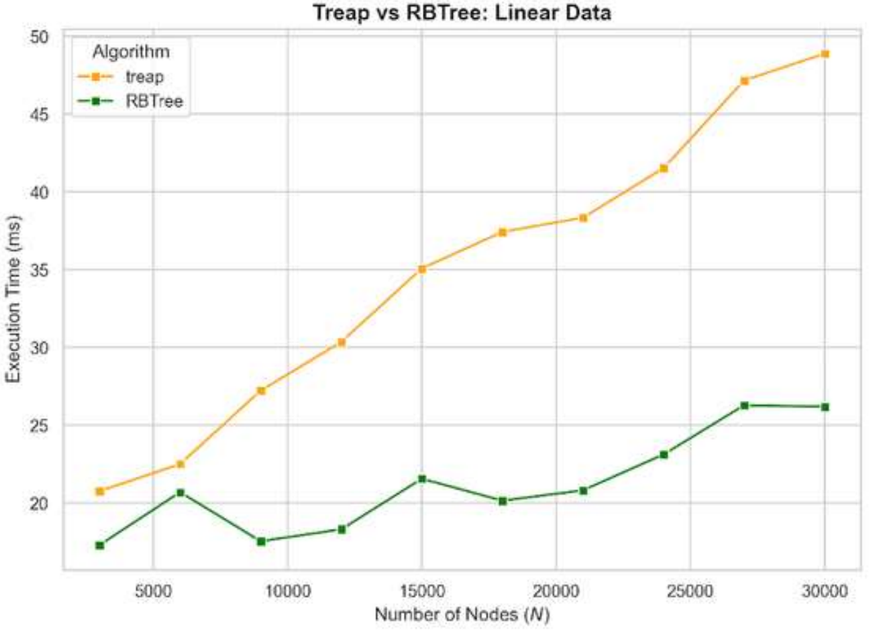
- Works well on random data
- Fails on sorted input
- Becomes ***O(n)*** in worst case

highly unreliable


Average Case (Random Data)
Algorithm
BST
treap
RBTree
Execution Time (ms)
Number of Nodes (N)


Worst Case (Linear/Sorted Data)
Algorithm
BST
treap
RBTree
Execution Time (ms)
Number of Nodes (N)

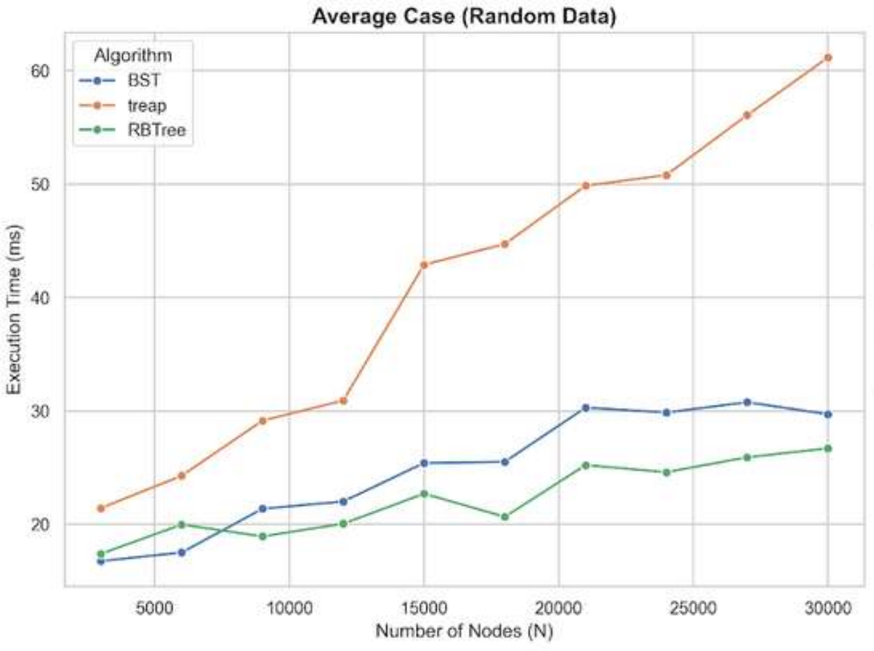

Treap vs RBTree: Random Data
Algorithm
treap
RBTree
Execution Time (ms)
Number of Nodes (N)

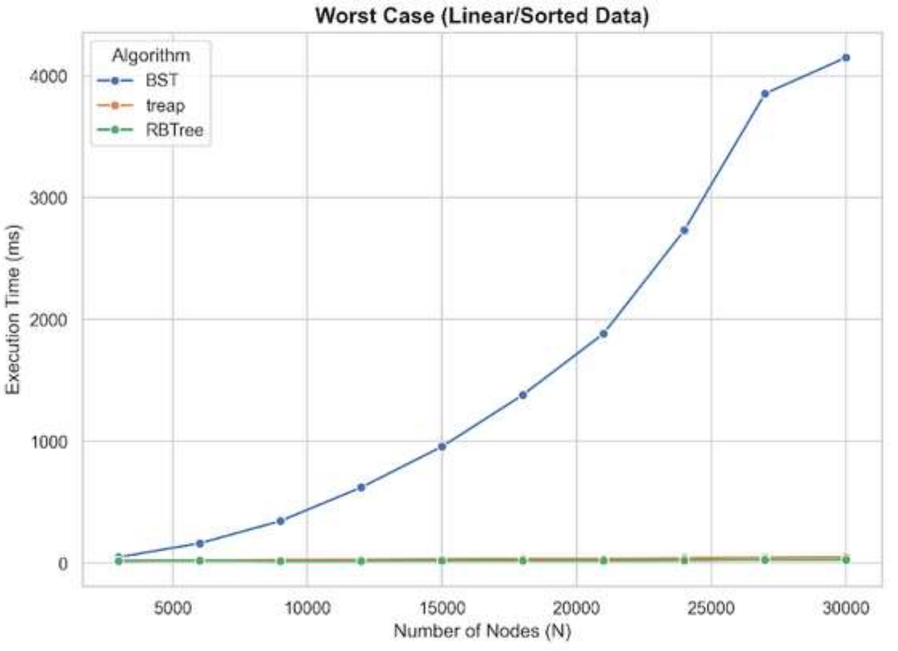

Treap vs RBTree: Linear Data
Algorithm
treap
RBTree
Execution Time (ms)
Number of Nodes (N)

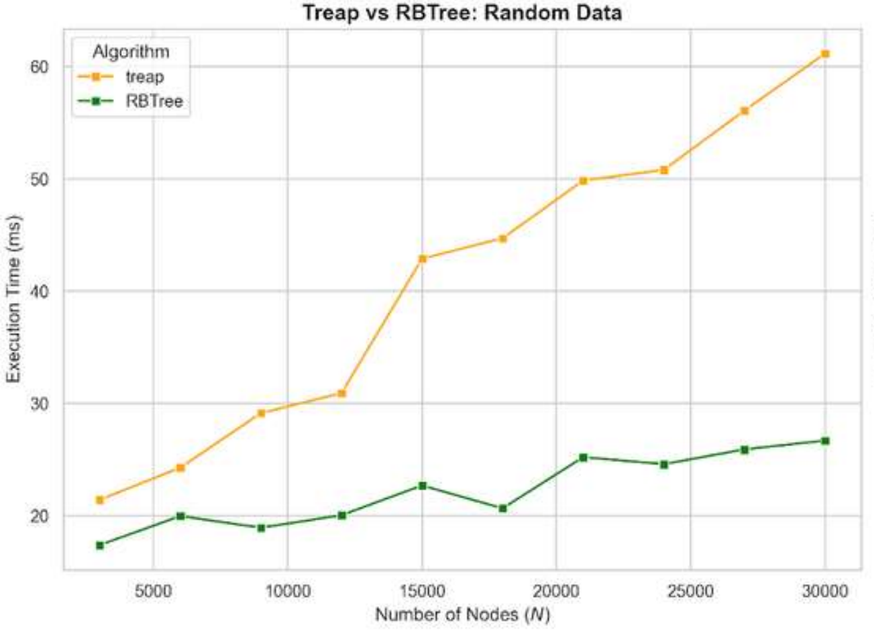
Treap Performance

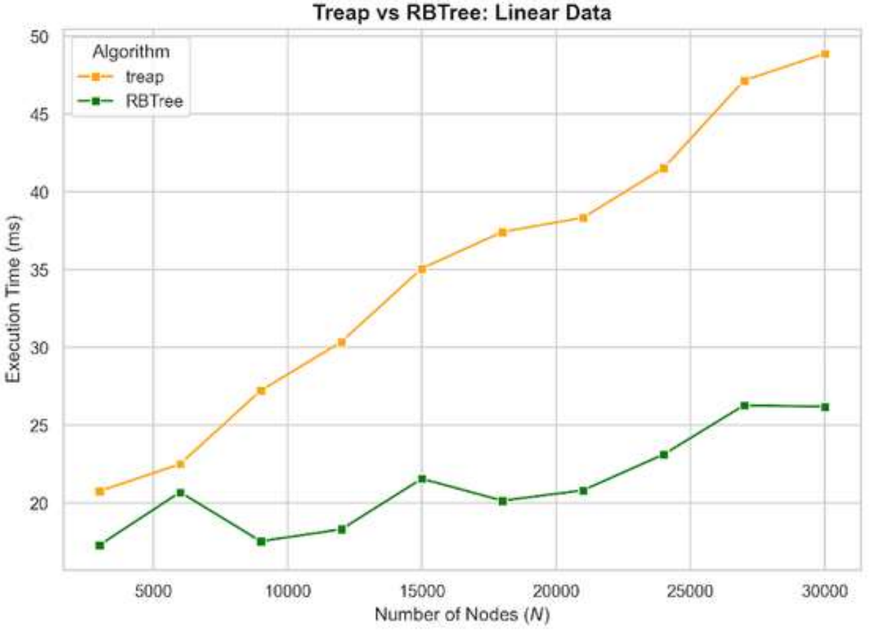
- Stable on all inputs
- Handles both:
  - random data
  - sorted data
- Slight overhead:
  - random priority
  - balancing operations

consistent performance


Average Case (Random Data)
Algorithm
BST
treap
RBTree
Execution Time (ms)
Number of Nodes (N)


Worst Case (Linear/Sorted Data)
Algorithm
BST
treap
RBTree
Execution Time (ms)
Number of Nodes (N)

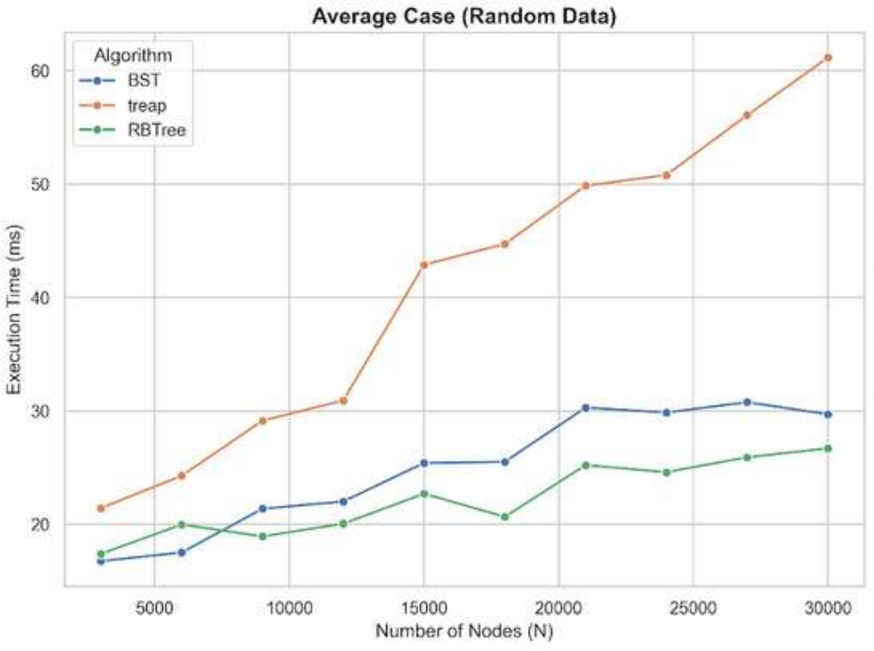

Treap vs RBTree: Random Data
Algorithm
treap
RBTree
Execution Time (ms)
Number of Nodes (N)

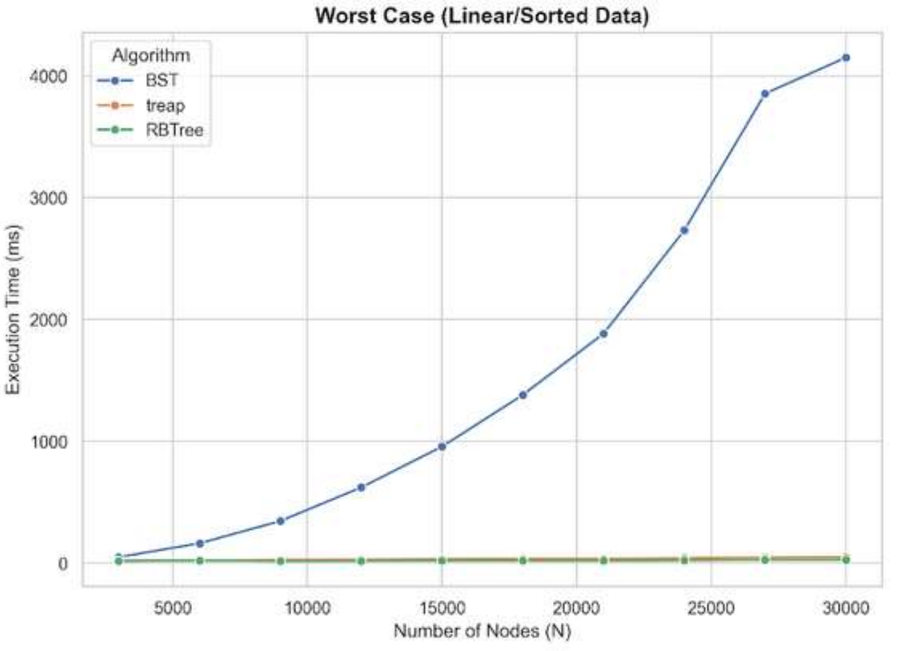

Treap vs RBTree: Linear Data
Algorithm
treap
RBTree
Execution Time (ms)
Number of Nodes (N)

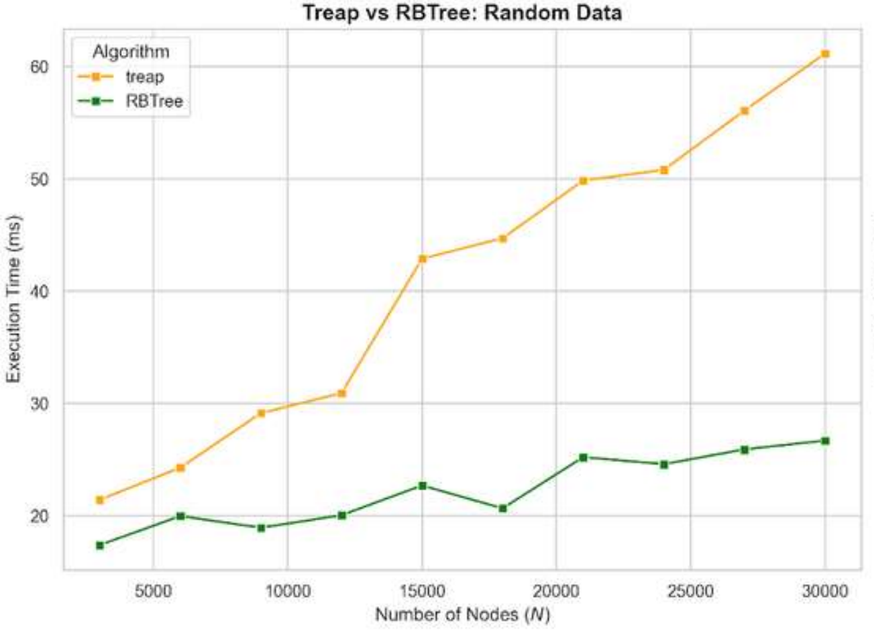
Red-Black Tree Performance

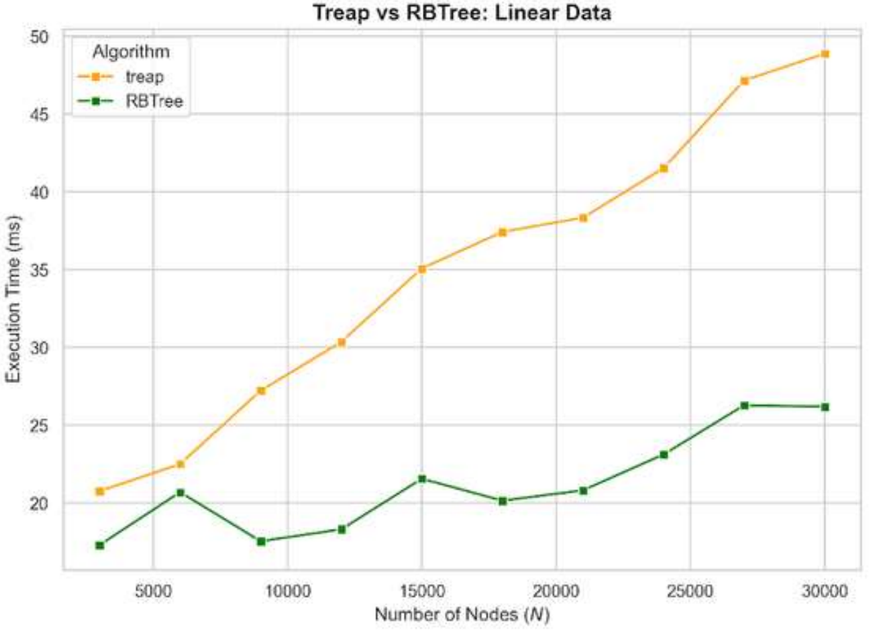
- Fastest in this benchmark
- Highly optimized balancing
- Very stable

But more complex to implement


Average Case (Random Data)
Algorithm
BST
treap
RBTree
Execution Time (ms)
Number of Nodes (N)

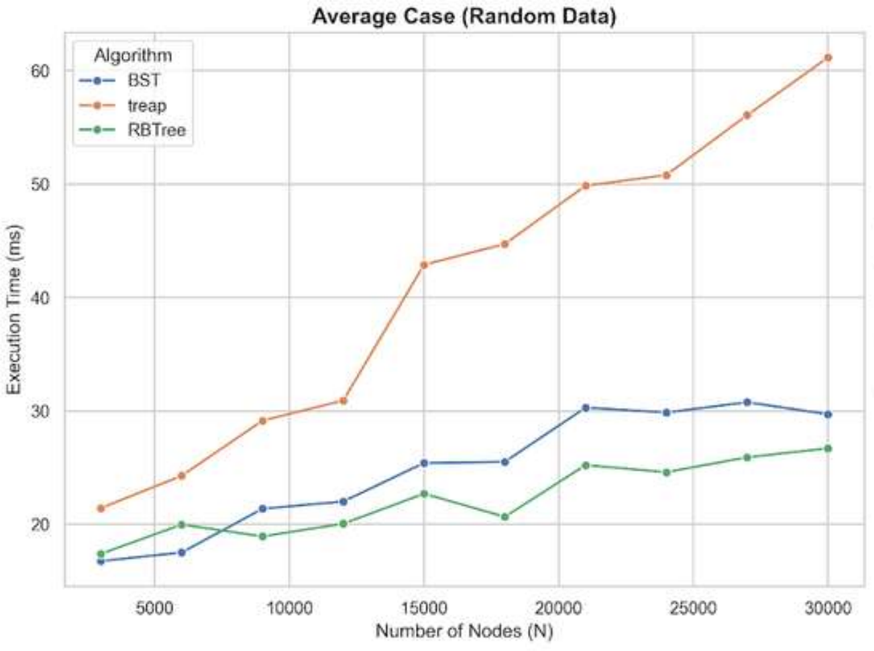

Worst Case (Linear/Sorted Data)
Algorithm
BST
treap
RBTree
Execution Time (ms)
Number of Nodes (N)

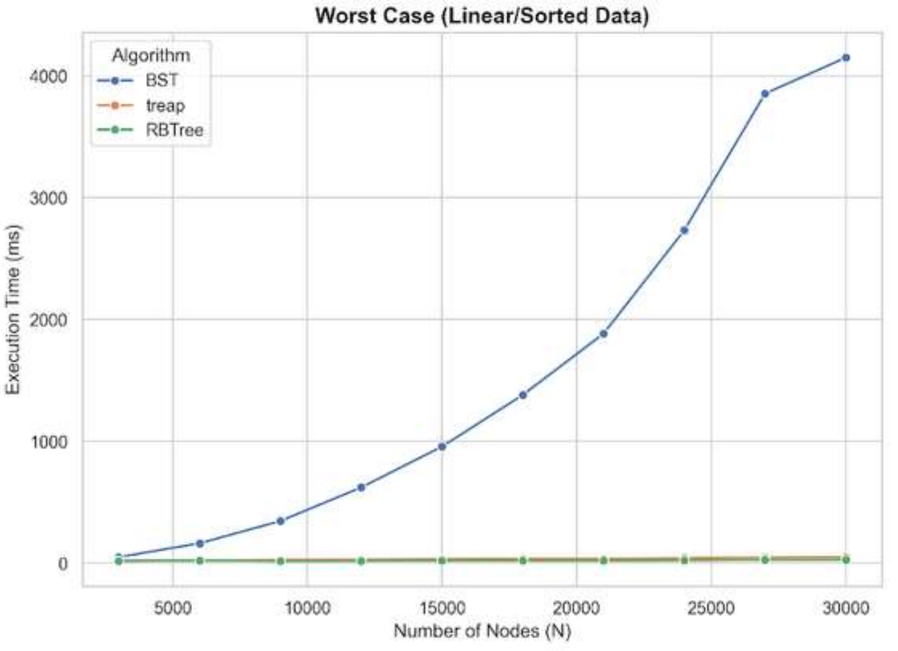

Treap vs RBTree: Random Data
Algorithm
treap
RBTree
Execution Time (ms)
Number of Nodes (N)

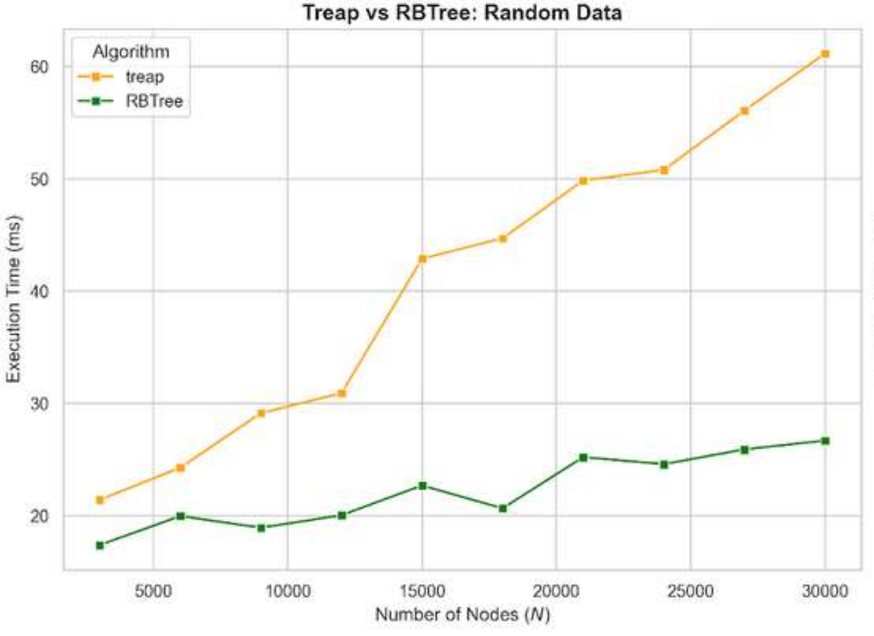

Treap vs RBTree: Linear Data
Algorithm
treap
RBTree
Execution Time (ms)
Number of Nodes (N)

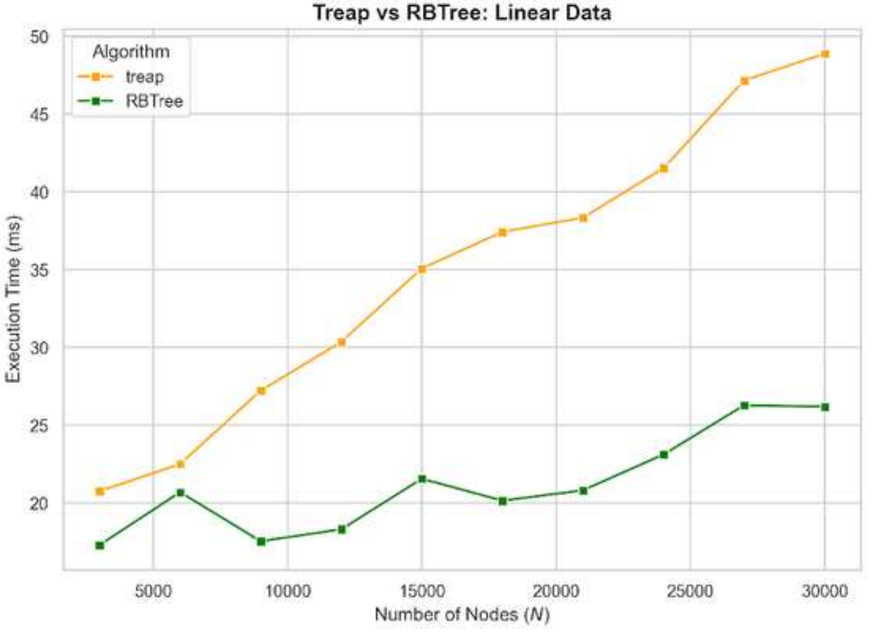
Treap vs Other Structures

- vs BST:
  - much more reliable
- vs Red-Black Tree:
  - slightly slower
  - but simpler and more flexible


Average Case (Random Data)
Algorithm
BST
treap
RBTree
Execution Time (ms)
Number of Nodes (N)


Worst Case (Linear/Sorted Data)
Algorithm
BST
treap
RBTree
Execution Time (ms)
Number of Nodes (N)


Treap vs RBTree: Random Data
Algorithm
treap
RBTree
Execution Time (ms)
Number of Nodes (N)


Treap vs RBTree: Linear Data
Algorithm
treap
RBTree
Execution Time (ms)
Number of Nodes (N)

Key Takeaways

- Treap is much better than brute force
  - More reliable than standard BST
  - Not always the fastest (RBTree wins)
- Best for:
  - range operations
  - dynamic sequences
  - flexible problems

Why Treap is Important

- Combines:
  - efficiency
  - flexibility
- Supports:
  - split
  - merge
  - range operations
- A powerful tool in advanced problems

Why Treap is Important

→ May not be the fastest structure, but offers a unique combination of simplicity, flexibility, and strong performance.

Quiz 1

Which two core operations are used to build most Treap algorithms?

- A. Push and Pop
- B. Search and Update
- C. Split and Merge
- D. Rotate and Recolor

Quiz 1

Which two core operations are used to build most Treap algorithms?

Answer:

- A. Push and Pop
- B. Search and Update
- C. Split and Merge
- D. Rotate and Recolor

Quiz 2

Why does a Treap tend to stay balanced?

- A. It performs rotations like AVL trees
- B. It enforces strict height constraints
- C. It uses random priorities
- D. It sorts nodes after insertion

Quiz 2

Why does a Treap tend to stay balanced?

Answer:

- A. It performs rotations like AVL trees
- B. It enforces strict height constraints
- C. It uses random priorities
- D. It sorts nodes after insertion

Quiz 3 — Merge condition

When can two treaps be merged directly?

- A. When both have equal size
- B. When their priorities are equal
- C. When all keys in left < all keys in right
- D. When both are balanced

Quiz 3 — Merge condition

When can two treaps be merged directly?

Answer:

- A. When both have equal size
- B. When their priorities are equal
- C. When all keys in left < all keys in right
- D. When both are balanced

Quiz 4

Which statement is TRUE about Treap?

- A. Treap always has lower time complexity than Red-Black Tree
- B. Treap guarantees perfect balance
- C. Treap has expected O(n log n) height
- D. Treap does depend on randomness

Quiz 4

Which statement is TRUE about Treap?

Answer:

- A. Treap always has lower time complexity than Red-Black Tree
- B. Treap guarantees perfect balance
- C. Treap has expected O(n log n) height
- D. Treap does depend on randomness

Quiz 5

If we insert sorted data into a Treap, what happens?

- A. Tree becomes a chain like BST
- B. Tree stays balanced (high probability)
- C. Tree becomes a heap
- D. It depends only on key values

Quiz 5

If we insert sorted data into a Treap, what happens?

Answer:

- A. Tree becomes a chain like BST
- B. Tree stays balanced (high probability)
- C. Tree becomes a heap
- D. It depends only on key values

- *“Treap ignores insertion order thanks to randomness.”*

Quiz 6

What happens if all priorities in a Treap are equal?

- A. Still balanced
- B. Becomes a standard BST
- C. Becomes a Linked List
- D. Program crashes

Quiz 6

What happens if all priorities in a Treap are equal?

Answer:

- A. Still balanced
- B. Becomes a standard BST
- C. Becomes a Linked List
- D. Program crashes

Treap

Thank you for listening